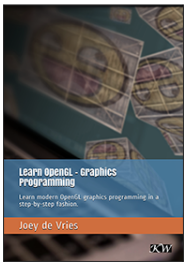
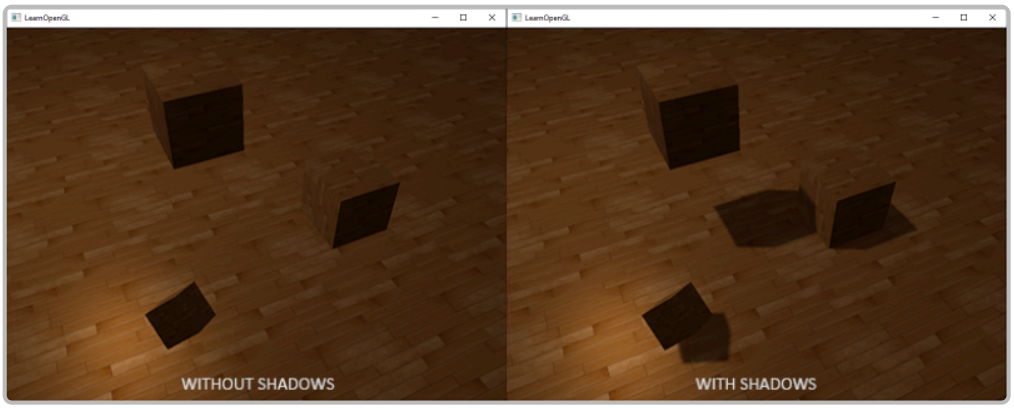
Introduction
Getting started
Lighting
Model Loading
Advanced OpenGL
Advanced Lighting
Advanced Lighting
Gamma Correction
Shadows
Shadow Mapping
Point Shadows
Normal Mapping
Parallax Mapping
HDR
Bloom
Deferred Shading
SSAO
PBR
In Practice
Guest Articles
Code repository
Translations
Privacy
About

PRINT EDITION



Shadow Mapping

Shadows are a result of the absence of light due to occlusion. When a light source's light rays do not hit an object because it gets occluded by some other object, the object is in shadow. Shadows add a great deal of realism to a lit scene and make it easier for a viewer to observe spatial relationships between objects. They give a greater sense of depth to our scene and objects. For example, take a look at the following image of a scene with and without shadows:



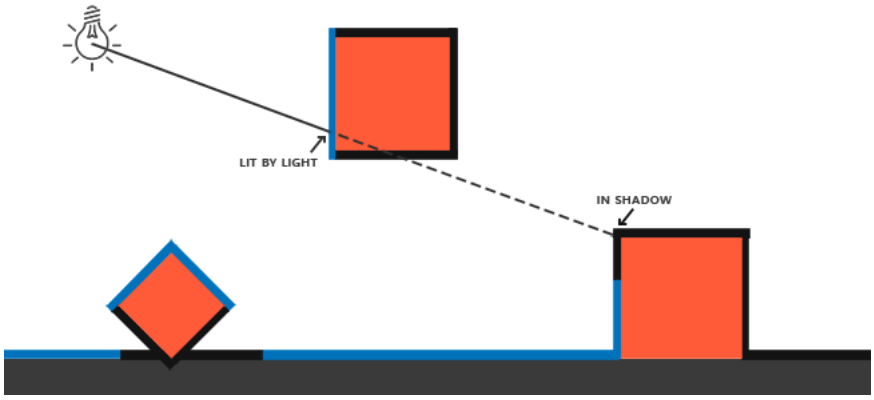
You can see that with shadows it becomes much more obvious how the objects relate to each other. For instance, the fact that one of the cubes is floating above the others is only really noticeable when we have shadows.

Shadows are a bit tricky to implement though, specifically because in current real-time (rasterized graphics) research a perfect shadow algorithm hasn't been developed yet. There are several good shadow approximation techniques, but they all have their little quirks and annoyances which we have to take into account.

One technique used by most videogames that gives decent results and is relatively easy to implement is shadow mapping. Shadow mapping is not too difficult to understand, doesn't cost too much in performance and quite easily extends into more advanced algorithms (like Omnidirectional Shadow Maps and Cascaded Shadow Maps).

Shadow mapping

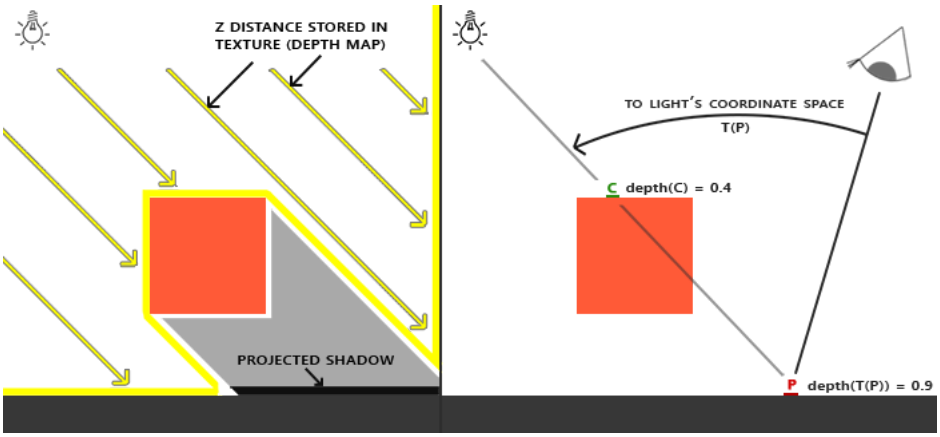
The idea behind shadow mapping is quite simple: we render the scene from the light's point of view and everything we see from the light's perspective is lit and everything we can't see must be in shadow. Imagine a floor section with a large box between itself and a light source. Since the light source will see this box and not the floor section when looking in its direction that specific floor section should be in shadow.



Here all the blue lines represent the fragments that the light source can see. The occluded fragments are shown as black lines: these are rendered as being shadowed. If we were to draw a line or ray from the light source to a fragment on the right-most box we can see the ray first hits the floating container before hitting the right-most container. As a result, the floating container's fragment is lit and the right-most container's

point and if so, the test point must be in shadow. Iterating through possibly thousands of light rays from such a light source is an extremely inefficient approach and doesn't lend itself too well for real-time rendering. We can do something similar, but without casting light rays. Instead, we use something we're quite familiar with: the depth buffer.

You may remember from the depth testing chapter that a value in the depth buffer corresponds to the depth of a fragment clamped to [0,1] from the camera's point of view. What if we were to render the scene from the light's perspective and store the resulting depth values in a texture? This way, we can sample the closest depth values as seen from the light's perspective. After all, the depth values show the first fragment visible from the light's perspective. We store all these depth values in a texture that we call a depth map or shadow map.



The left image shows a directional light source (all light rays are parallel) casting a shadow on the surface below the cube. Using the depth values stored in the depth map we find the closest point and use that to determine whether fragments are in shadow. We create the depth map by rendering the scene (from the light's perspective) using a view and projection matrix specific to that light source. This projection and view matrix together form a transformation T that transforms any 3D position to the light's (visible) coordinate space.

> A directional light doesn't have a position as it's modelled to be infinitely far away. However, for the sake of shadow mapping we need to render the scene from a light's perspective and thus render the scene from a position somewhere along the lines of the light direction.

In the right image we see the same directional light and the viewer. We render a fragment at point $\bar{P}$ for which we have to determine whether it is in shadow. To do this, we first transform point $\bar{P}$ to the light's coordinate space using T. Since point $\bar{P}$ is now as seen from the light's perspective, its `z` coordinate corresponds to its depth which in this example is `0.9`. Using point $\bar{P}$ we can also index the depth/shadow map to obtain the closest visible depth from the light's perspective, which is at point $\bar{C}$ with a sampled depth of `0.4`. Since indexing the depth map returns a depth smaller than the depth at point $\bar{P}$ we can conclude point $\bar{P}$ is occluded and thus in shadow.

Shadow mapping therefore consists of two passes: first we render the depth map, and in the second pass we render the scene as normal and use the generated depth map to calculate whether fragments are in shadow. It may sound a bit complicated, but as soon as we walk through the technique step-by-step it'll likely start to make sense.

The depth map

The first pass requires us to generate a depth map. The depth map is the depth texture as rendered from the light's perspective that we'll be using for testing for shadows. Because we need to store the rendered result of a scene into a texture we're going to need framebuffers again.

First we'll create a framebuffer object for rendering the depth map:

```
unsigned int depthMapFBO;
glGenFramebuffers(1, &depthMapFBO);
```

Next we create a 2D texture that we'll use as the framebuffer's depth buffer:

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT,
             SHADOW_WIDTH, SHADOW_HEIGHT, 0, GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

Generating the depth map shouldn't look too complicated. Because we only care about depth values we specify the texture's formats as `GL_DEPTH_COMPONENT`. We also give the texture a width and height of `1024`: this is the resolution of the depth map.

With the generated depth texture we can attach it as the framebuffer's depth buffer:

```
glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO);
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, depthMap, 0);
glDrawBuffer(GL_NONE);
glReadBuffer(GL_NONE);
glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

We only need the depth information when rendering the scene from the light's perspective so there is no need for a color buffer. A framebuffer object however is not complete without a color buffer so we need to explicitly tell OpenGL we're not going to render any color data. We do this by setting both the read and draw buffer to `GL_NONE` with `glDrawBuffer` and `glReadbuffer`.

With a properly configured framebuffer that renders depth values to a texture we can start the first pass: generate the depth map. When combined with the second pass, the complete rendering stage will look a bit like this:

```
// 1. first render to depth map
glViewport(0, 0, SHADOW_WIDTH, SHADOW_HEIGHT);
glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO);
    glClear(GL_DEPTH_BUFFER_BIT);
    ConfigureShaderAndMatrices();
    RenderScene();
glBindFramebuffer(GL_FRAMEBUFFER, 0);
// 2. then render scene as normal with shadow mapping (using depth map)
glViewport(0, 0, SCR_WIDTH, SCR_HEIGHT);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
ConfigureShaderAndMatrices();
glBindTexture(GL_TEXTURE_2D, depthMap);
RenderScene();
```

This code left out some details, but it'll give you the general idea of shadow mapping. What is important to note here are the calls to `glViewport`. Because shadow maps often have a different resolution compared to what we originally render the scene in (usually the window resolution), we need to change the viewport parameters to accommodate for the size of the shadow map. If we forget to update the viewport parameters, the resulting depth map will be either incomplete or too small.

Light space transform

An unknown in the previous snippet of code is the `ConfigureShaderAndMatrices` function. In the second pass this is business as usual: make sure proper projection and view matrices are set, and set the relevant model matrices per object. However, in the first pass we need to use a different projection and view matrix to render the scene from the light's point of view.

Because we're modelling a directional light source, all its light rays are parallel. For this reason, we're going to use an orthographic projection matrix for the light source where there is no perspective deform:

```
float near_plane = 1.0f, far_plane = 7.5f;
glm::mat4 lightProjection = glm::ortho(-10.0f, 10.0f, -10.0f, 10.0f, near_plane, far_plar
```

Here is an example orthographic projection matrix as used in this chapter's demo scene. Because a projection matrix indirectly determines the range of what is visible (e.g. what is not clipped) you want to make sure the size of the projection frustum correctly contains the objects you want to be in the depth map. When objects or fragments are not in the depth map they will not produce shadows.

To create a view matrix to transform each object so they're visible from the light's point of view, we're going to

```
glm::mat4 lightView = glm::lookAt(glm::vec3(-2.0f, 4.0f, -1.0f),
                                  glm::vec3( 0.0f, 0.0f,  0.0f),
                                  glm::vec3( 0.0f, 1.0f,  0.0f));
```

Combining these two gives us a light space transformation matrix that transforms each world-space vector into the space as visible from the light source; exactly what we need to render the depth map.

```
glm::mat4 lightSpaceMatrix = lightProjection * lightView;
```

This `lightSpaceMatrix` is the transformation matrix that we earlier denoted as T. With this `lightSpaceMatrix`, we can render the scene as usual as long as we give each shader the light-space equivalents of the projection and view matrices. However, we only care about depth values and not all the expensive fragment (lighting) calculations. To save performance we're going to use a different, but much simpler shader for rendering to the depth map.

Render to depth map

When we render the scene from the light's perspective we'd much rather use a simple shader that only transforms the vertices to light space and not much more. For such a simple shader called `simpleDepthShader` we'll use the following vertex shader:

```
#version 330 core
layout (location = 0) in vec3 aPos;

uniform mat4 lightSpaceMatrix;
uniform mat4 model;

void main()
{
    gl_Position = lightSpaceMatrix * model * vec4(aPos, 1.0);
}
```

This vertex shader takes a per-object model, a vertex, and transforms all vertices to light space using `lightSpaceMatrix`.

Since we have no color buffer and disabled the draw and read buffers, the resulting fragments do not require any processing so we can simply use an empty fragment shader:

```
#version 330 core

void main()
{
    // gl_FragDepth = gl_FragCoord.z;
}
```

This empty fragment shader does no processing whatsoever, and at the end of its run the depth buffer is updated. We could explicitly set the depth by uncommenting its one line, but this is effectively what happens behind the scene anyways.

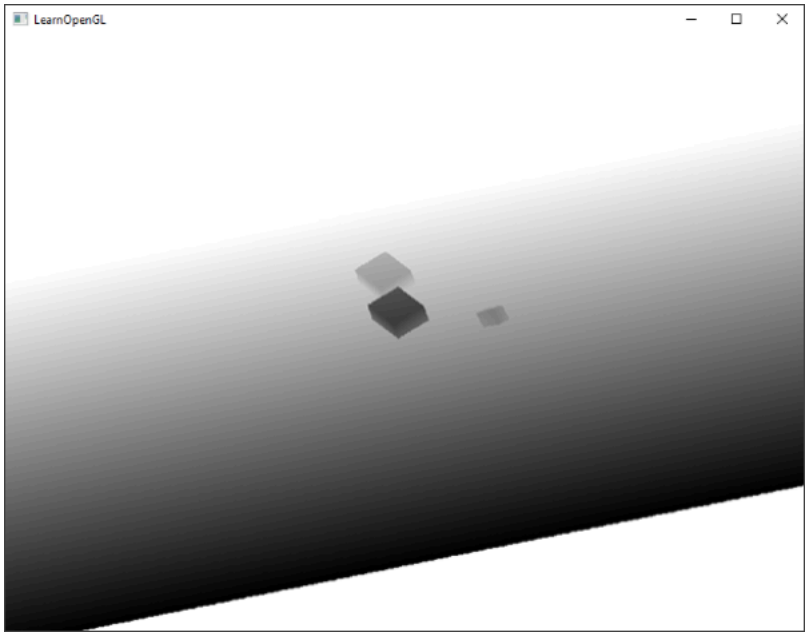
Rendering the depth/shadow map now effectively becomes:

```
simpleDepthShader.use();
glUniformMatrix4fv(lightSpaceMatrixLocation, 1, GL_FALSE, glm::value_ptr(lightSpaceMatrix

glViewport(0, 0, SHADOW_WIDTH, SHADOW_HEIGHT);
glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO);
    glClear(GL_DEPTH_BUFFER_BIT);
    RenderScene(simpleDepthShader);
glBindFramebuffer(GL_FRAMEBUFFER, 0);
```

Here the `RenderScene` function takes a shader program, calls all relevant drawing functions and sets the corresponding model matrices where necessary.

The result is a nicely filled depth buffer holding the closest depth of each visible fragment from the light's perspective. By rendering this texture onto a 2D quad that fills the screen (similar to what we did in the post-



For rendering the depth map onto a quad we used the following fragment shader:

```
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D depthMap;

void main()
{
    float depthValue = texture(depthMap, TexCoords).r;
    FragColor = vec4(vec3(depthValue), 1.0);
}
```

Note that there are some subtle changes when displaying depth using a perspective projection matrix instead of an orthographic projection matrix as depth is non-linear when using perspective projection. At the end of this chapter we'll discuss some of these subtle differences.

You can find the source code for rendering a scene to a depth map here.

Rendering shadows

With a properly generated depth map we can start rendering the actual shadows. The code to check if a fragment is in shadow is (quite obviously) executed in the fragment shader, but we do the light-space transformation in the vertex shader:

```
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal;
layout (location = 2) in vec2 aTexCoords;

out VS_OUT {
    vec3 FragPos;
    vec3 Normal;
    vec2 TexCoords;
    vec4 FragPosLightSpace;
} vs_out;

uniform mat4 projection;
uniform mat4 view;
uniform mat4 model;
uniform mat4 lightSpaceMatrix;

void main()
{
```

```
    gl_Position = projection * view * vec4(vs_out.FragPos, 1.0);
}
```

What is new here is the extra output vector `FragPosLightSpace`. We take the same `lightSpaceMatrix` (used to transform vertices to light space in the depth map stage) and transform the world-space vertex position to light space for use in the fragment shader.

The main fragment shader we'll use to render the scene uses the Blinn-Phong lighting model. Within the fragment shader we then calculate a `shadow` value that is either `1.0` when the fragment is in shadow or `0.0` when not in shadow. The resulting `diffuse` and `specular` components are then multiplied by this shadow component. Because shadows are rarely completely dark (due to light scattering) we leave the `ambient` component out of the shadow multiplications.

```
#version 330 core
out vec4 FragColor;

in VS_OUT {
    vec3 FragPos;
    vec3 Normal;
    vec2 TexCoords;
    vec4 FragPosLightSpace;
} fs_in;

uniform sampler2D diffuseTexture;
uniform sampler2D shadowMap;

uniform vec3 lightPos;
uniform vec3 viewPos;

float ShadowCalculation(vec4 fragPosLightSpace)
{
    [...]
}

void main()
{
    vec3 color = texture(diffuseTexture, fs_in.TexCoords).rgb;
    vec3 normal = normalize(fs_in.Normal);
    vec3 lightColor = vec3(1.0);
    // ambient
    vec3 ambient = 0.15 * lightColor;
    // diffuse
    vec3 lightDir = normalize(lightPos - fs_in.FragPos);
    float diff = max(dot(lightDir, normal), 0.0);
    vec3 diffuse = diff * lightColor;
    // specular
    vec3 viewDir = normalize(viewPos - fs_in.FragPos);
    float spec = 0.0;
    vec3 halfwayDir = normalize(lightDir + viewDir);
    spec = pow(max(dot(normal, halfwayDir), 0.0), 64.0);
    vec3 specular = spec * lightColor;
    // calculate shadow
    float shadow = ShadowCalculation(fs_in.FragPosLightSpace);
    vec3 lighting = (ambient + (1.0 - shadow) * (diffuse + specular)) * color;

    FragColor = vec4(lighting, 1.0);
}
```

The fragment shader is largely a copy from what we used in the advanced lighting chapter, but with an added shadow calculation. We declared a function `ShadowCalculation` that does most of the shadow work. At the end of the fragment shader, we multiply the diffuse and specular contributions by the inverse of the `shadow` component e.g. how much the fragment is *not* in shadow. This fragment shader takes as extra input the light-space fragment position and the depth map generated from the first render pass.

The first thing to do to check whether a fragment is in shadow, is transform the light-space fragment position in clip-space to normalized device coordinates. When we output a clip-space vertex position to `gl_Position` in the vertex shader, OpenGL automatically does a perspective divide e.g. transform clip-space coordinates in the range `[-w,w]` to `[-1,1]` by dividing the `x`, `y` and `z` component by the vector's `w` component. As the clip-space `FragPosLightSpace` is not passed to the fragment shader through `gl_Position`, we have to do this perspective divide ourselves:

```
    [...]
}
```

This returns the fragment's light-space position in the range [-1,1].

> When using an orthographic projection matrix the w component of a vertex remains untouched so this step is actually quite meaningless. However, it is necessary when using perspective projection so keeping this line ensures it works with both projection matrices.

Because the depth from the depth map is in the range [0,1] and we also want to use `projCoords` to sample from the depth map, we transform the NDC coordinates to the range [0,1]:

```
projCoords = projCoords * 0.5 + 0.5;
```

With these projected coordinates we can sample the depth map as the resulting [0,1] coordinates from `projCoords` directly correspond to the transformed NDC coordinates from the first render pass. This gives us the closest depth from the light's point of view:

```
float closestDepth = texture(shadowMap, projCoords.xy).r;
```

To get the current depth at this fragment we simply retrieve the projected vector's z coordinate which equals the depth of this fragment from the light's perspective.

```
float currentDepth = projCoords.z;
```

The actual comparison is then simply a check whether `currentDepth` is higher than `closestDepth` and if so, the fragment is in shadow:

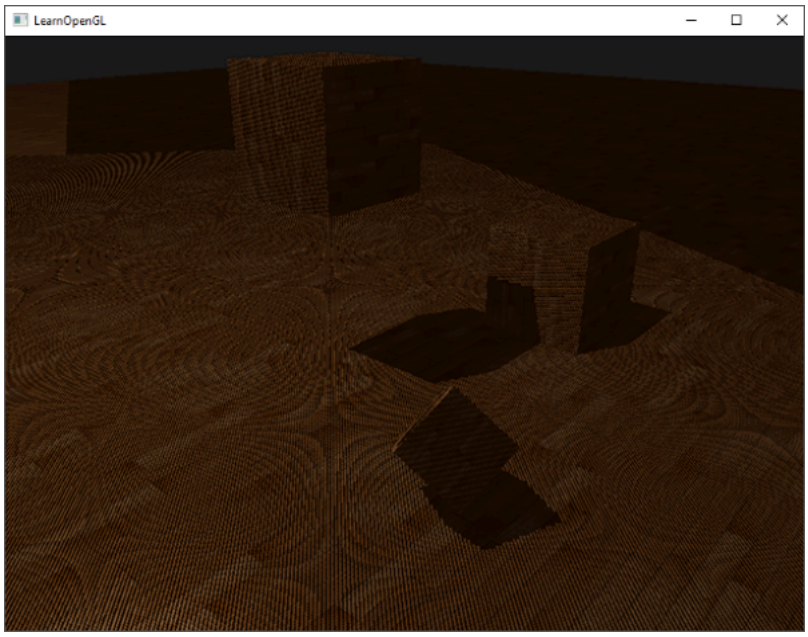
```
float shadow = currentDepth > closestDepth  ? 1.0 : 0.0;
```

The complete `ShadowCalculation` function then becomes:

```
float ShadowCalculation(vec4 fragPosLightSpace)
{
    // perform perspective divide
    vec3 projCoords = fragPosLightSpace.xyz / fragPosLightSpace.w;
    // transform to [0,1] range
    projCoords = projCoords * 0.5 + 0.5;
    // get closest depth value from light's perspective (using [0,1] range fragPosLight a
    float closestDepth = texture(shadowMap, projCoords.xy).r;
    // get depth of current fragment from light's perspective
    float currentDepth = projCoords.z;
    // check whether current frag pos is in shadow
    float shadow = currentDepth > closestDepth  ? 1.0 : 0.0;

    return shadow;
}
```

Activating this shader, binding the proper textures, and activating the default projection and view matrices in the second render pass should give you a result similar to the image below:



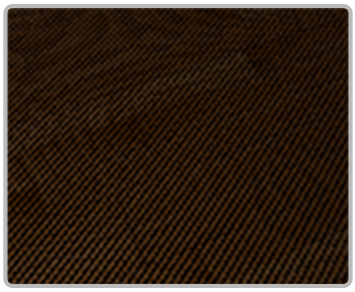
If you did things right you should indeed see (albeit with quite a few artifacts) shadows on the floor and the cubes. You can find the source code of the demo application here.

Improving shadow maps

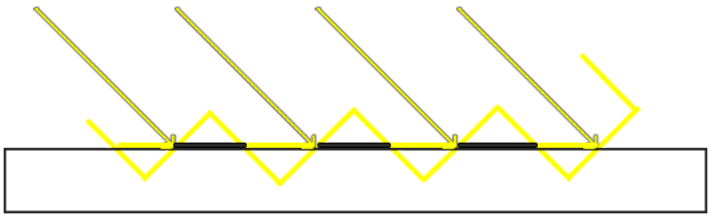
We managed to get the basics of shadow mapping working, but as you can we're not there yet due to several (clearly visible) artifacts related to shadow mapping we need to fix. We'll focus on fixing these artifacts in the next sections.

Shadow acne

It is obvious something is wrong from the previous image. A closer zoom shows us a very obvious Moiré-like pattern:



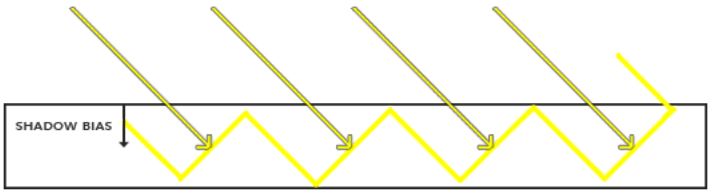
We can see a large part of the floor quad rendered with obvious black lines in an alternating fashion. This shadow mapping artifact is called shadow acne and can be explained by the following image:



Because the shadow map is limited by resolution, multiple fragments can sample the same value from the depth map when they're relatively far away from the light source. The image shows the floor where each yellow tilted panel represents a single texel of the depth map. As you can see, several fragments sample the same depth sample.

While this is generally okay, it becomes an issue when the light source looks at an angle towards the surface as in that case the depth map is also rendered from an angle. Several fragments then access the same tilted depth texel while some are above and some below the floor; we get a shadow discrepancy. Because of this

above the surface.



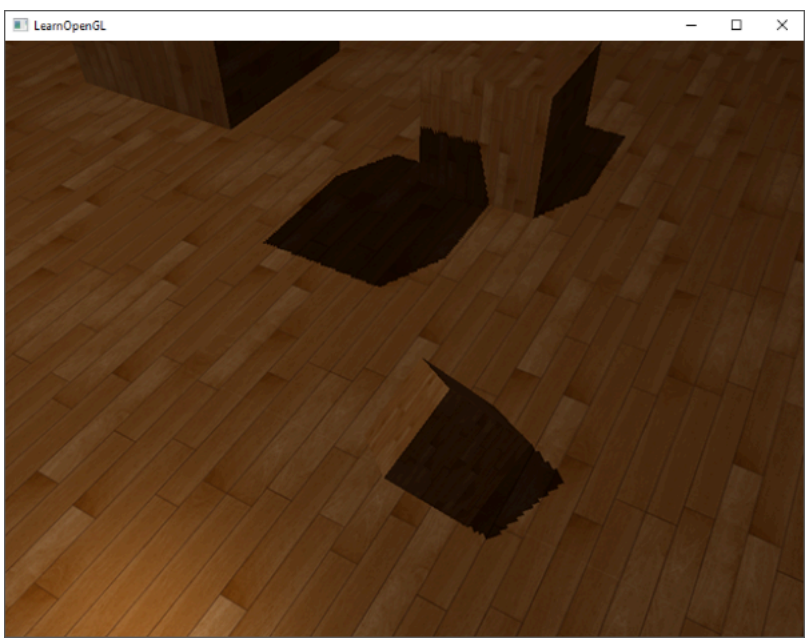
With the bias applied, all the samples get a depth smaller than the surface's depth and thus the entire surface is correctly lit without any shadows. We can implement such a bias as follows:

```
float bias = 0.005;
float shadow = currentDepth - bias > closestDepth  ? 1.0 : 0.0;
```

A shadow bias of `0.005` solves the issues of our scene by a large extent, but you can imagine the bias value is highly dependent on the angle between the light source and the surface. If the surface would have a steep angle to the light source, the shadows may still display shadow acne. A more solid approach would be to change the amount of bias based on the surface angle towards the light: something we can solve with the dot product:

```
float bias = max(0.05 * (1.0 - dot(normal, lightDir)), 0.005);
```

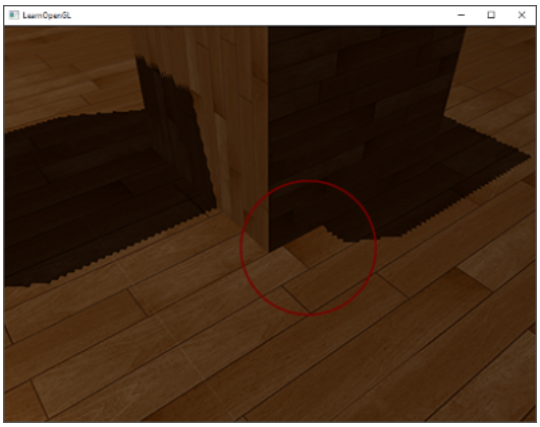
Here we have a maximum bias of `0.05` and a minimum of `0.005` based on the surface's normal and light direction. This way, surfaces like the floor that are almost perpendicular to the light source get a small bias, while surfaces like the cube's side-faces get a much larger bias. The following image shows the same scene but now with a shadow bias:



Choosing the correct bias value(s) requires some tweaking as this will be different for each scene, but most of the time it's simply a matter of slowly incrementing the bias until all acne is removed.

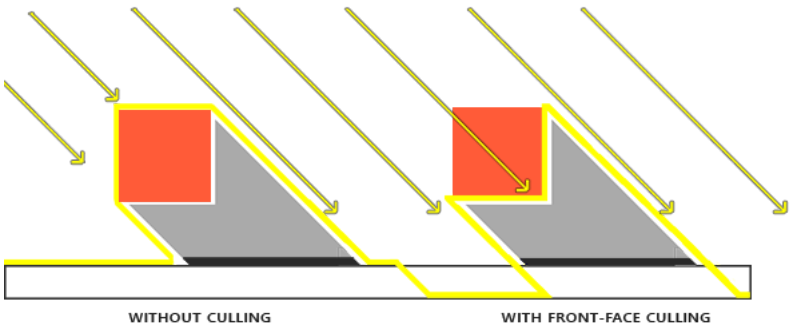
Peter panning

A disadvantage of using a shadow bias is that you're applying an offset to the actual depth of objects. As a result, the bias may become large enough to see a visible offset of shadows compared to the actual object locations as you can see below (with an exaggerated bias value):



This shadow artifact is called peter panning since objects seem slightly *detached* from their shadows. We can use a little trick to solve most of the peter panning issue by using front face culling when rendering the depth map. You may remember from the face culling chapter that OpenGL by default culls back-faces. By telling OpenGL we want to cull front faces during the shadow map stage we're switching that order around.

Because we only need depth values for the depth map it shouldn't matter for solid objects whether we take the depth of their front faces or their back faces. Using their back face depths doesn't give wrong results as it doesn't matter if we have shadows inside objects; we can't see there anyways.



To fix peter panning we cull all front faces during the shadow map generation. Note that you need to enable `GL_CULL_FACE` first.

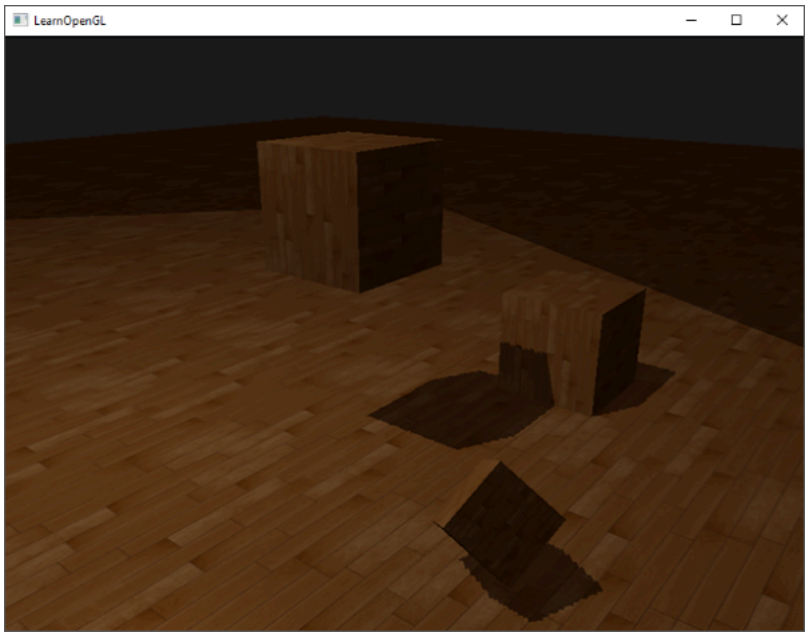
```
glCullFace(GL_FRONT);
RenderSceneToDepthMap();
glCullFace(GL_BACK); // don't forget to reset original culling face
```

This effectively solves the peter panning issues, but only for solid objects that actually have an inside without openings. In our scene for example, this works perfectly fine on the cubes. However, on the floor it won't work as well as culling the front face completely removes the floor from the equation. The floor is a single plane and would thus be completely culled. If one wants to solve peter panning with this trick, care has to be taken to only cull the front faces of objects where it makes sense.

Another consideration is that objects that are close to the shadow receiver (like the distant cube) may still give incorrect results. However, with normal bias values you can generally avoid peter panning.

Over sampling

Another visual discrepancy which you may like or dislike is that regions outside the light's visible frustum are considered to be in shadow while they're (usually) not. This happens because projected coordinates outside the light's frustum are higher than `1.0` and will thus sample the depth texture outside its default range of [`0`,`1`]. Based on the texture's wrapping method, we will get incorrect depth results not based on the real depth values from the light source.

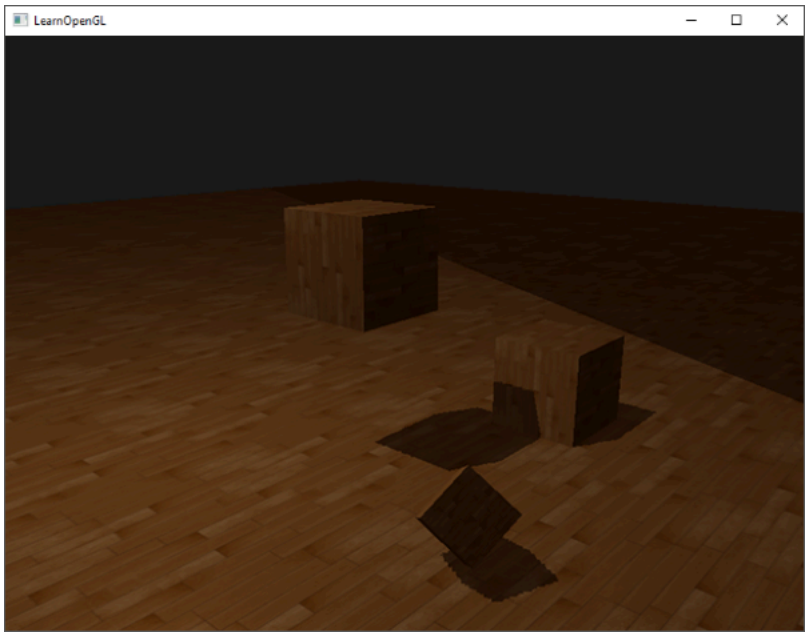


You can see in the image that there is some sort of imaginary region of light, and a large part outside this area is in shadow; this area represents the size of the depth map projected onto the floor. The reason this happens is that we earlier set the depth map's wrapping options to `GL_REPEAT`.

What we'd rather have is that all coordinates outside the depth map's range have a depth of `1.0` which as a result means these coordinates will never be in shadow (as no object will have a depth larger than `1.0`). We can do this by configuring a texture border color and set the depth map's texture wrap options to `GL_CLAMP_TO_BORDER`:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_BORDER);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_BORDER);
float borderColor[] = { 1.0f, 1.0f, 1.0f, 1.0f };
glTexParameterfv(GL_TEXTURE_2D, GL_TEXTURE_BORDER_COLOR, borderColor);
```

Now whenever we sample outside the depth map's [`0`,`1`] coordinate range, the `texture` function will always return a depth of `1.0`, producing a `shadow` value of `0.0`. The result now looks more plausible:



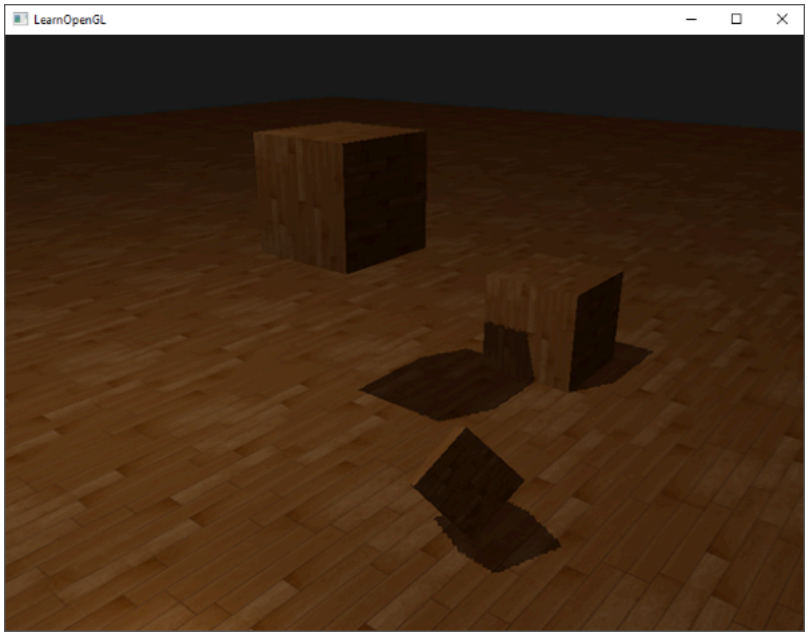
There seems to still be one part showing a dark region. Those are the coordinates outside the far plane of the light's orthographic frustum. You can see that this dark region always occurs at the far end of the light source's frustum by looking at the shadow directions.

The fix for this is also relatively easy as we simply force the `shadow` value to `0.0` whenever the projected vector's `z` coordinate is larger than `1.0`:

```
float ShadowCalculation(vec4 fragPosLightSpace)
{
    [...]
    if(projCoords.z > 1.0)
        shadow = 0.0;

    return shadow;
}
```

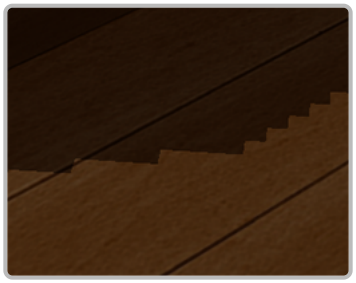
Checking the far plane and clamping the depth map to a manually specified border color solves the over-sampling of the depth map. This finally gives us the result we are looking for:



The result of all this does mean that we only have shadows where the projected fragment coordinates sit inside the depth map range so anything outside the light frustum will have no visible shadows. As games usually make sure this only occurs in the distance it is a much more plausible effect than the obvious black regions we had before.

PCF

The shadows right now are a nice addition to the scenery, but it's still not exactly what we want. If you were to zoom in on the shadows the resolution dependency of shadow mapping quickly becomes apparent.



Because the depth map has a fixed resolution, the depth frequently usually spans more than one fragment per texel. As a result, multiple fragments sample the same depth value from the depth map and come to the same shadow conclusions, which produces these jagged blocky edges.

You can reduce these blocky shadows by increasing the depth map resolution, or by trying to fit the light frustum as closely to the scene as possible.

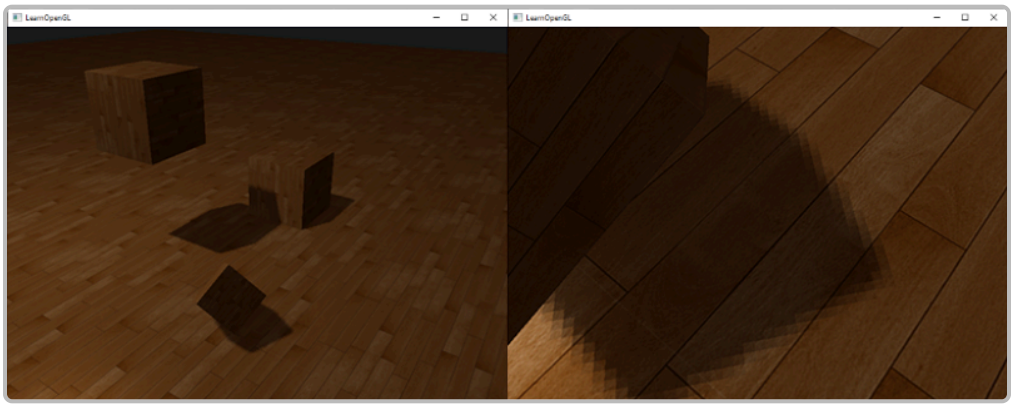
Another (partial) solution to these jagged edges is called PCF, or percentage-closer filtering, which is a term that hosts many different filtering functions that produce *softer* shadows, making them appear less blocky or

One simple implementation of PCF is to simply sample the surrounding texels of the depth map and average the results:

```
float shadow = 0.0;
vec2 texelSize = 1.0 / textureSize(shadowMap, 0);
for(int x = -1; x <= 1; ++x)
{
    for(int y = -1; y <= 1; ++y)
    {
        float pcfDepth = texture(shadowMap, projCoords.xy + vec2(x, y) * texelSize).r;
        shadow += currentDepth - bias > pcfDepth ? 1.0 : 0.0;
    }
}
shadow /= 9.0;
```

Here `textureSize` returns a `vec2` of the width and height of the given sampler texture at mipmap level `0`. 1 divided over this returns the size of a single texel that we use to offset the texture coordinates, making sure each new sample samples a different depth value. Here we sample 9 values around the projected coordinate's `x` and `y` value, test for shadow occlusion, and finally average the results by the total number of samples taken.

By using more samples and/or varying the `texelSize` variable you can increase the quality of the soft shadows. Below you can see the shadows with simple PCF applied:



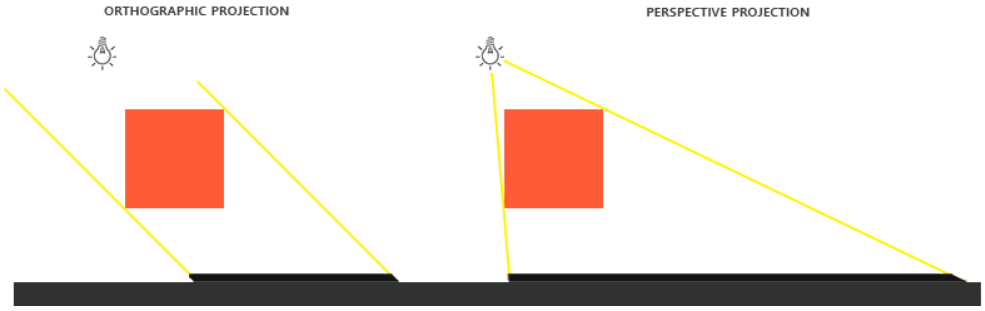
From a distance the shadows look a lot better and less hard. If you zoom in you can still see the resolution artifacts of shadow mapping, but in general this gives good results for most applications.

You can find the complete source code of the example here.

There is actually much more to PCF and quite a few techniques to considerably improve the quality of soft shadows, but for the sake of this chapter's length we'll leave that for a later discussion.

Orthographic vs perspective

There is a difference between rendering the depth map with an orthographic or a perspective projection matrix. An orthographic projection matrix does not deform the scene with perspective so all view/light rays are parallel. This makes it a great projection matrix for directional lights. A perspective projection matrix however does deform all vertices based on perspective which gives different results. The following image shows the different shadow regions of both projection methods:



Perspective projections make most sense for light sources that have actual locations, unlike directional lights. Perspective projections are most often used with spotlights and point lights, while orthographic projections are used for directional lights.

properly view the depth values as we did with the orthographic projection you first want to transform the non-linear depth values to linear as we discussed in the depth testing chapter:

```
#version 330 core
out vec4 FragColor;

in vec2 TexCoords;

uniform sampler2D depthMap;
uniform float near_plane;
uniform float far_plane;

float LinearizeDepth(float depth)
{
    float z = depth * 2.0 - 1.0; // Back to NDC
    return (2.0 * near_plane * far_plane) / (far_plane + near_plane - z * (far_plane - ne
}

void main()
{
    float depthValue = texture(depthMap, TexCoords).r;
    FragColor = vec4(vec3(LinearizeDepth(depthValue) / far_plane), 1.0); // perspective
    // FragColor = vec4(vec3(depthValue), 1.0); // orthographic
}
```

This shows depth values similar to what we've seen with orthographic projection. Note that this is only useful for debugging; the depth checks remain the same with orthographic or projection matrices as the relative depths do not change.

Additional resources

- Tutorial 16 : Shadow mapping: similar shadow mapping tutorial by opengl-tutorial.org with a few extra notes.
- Shadow Mapping - Part 1: another shadow mapping tutorial by ogldev.
- How Shadow Mapping Works: a 3-part YouTube tutorial by TheBennyBox on shadow mapping and its implementation.
- Common Techniques to Improve Shadow Depth Maps: a great article by Microsoft listing a large number of techniques to improve the quality of shadow maps.
- How I Implemented Shadows in my Game Engine: great video by ThinMatrix on his methods of improving shadow maps.

232 Comments

Login ▾

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

25 Share

Best Newest Oldest

Hermes Mercurio

10 years ago

"This shadow artifact is called peter panning since objects seem to slightly hover above the surface."

No, I think it's because they appear detached from their shadows. Just like Peter Pan who at some point lost his shadow, which was later found by Wendy and sewn back to his feet. It has nothing to do with hovering.

see more

28 0 Reply

daf → Hermes Mercurio

7 years ago

So why disapper? Because of hovering.

0 0 Reply

Arccch

9 years ago

Hi, thank you very much for your lessons. I would like to share some of my experience of shadowmap implementig. When I was dealing with peter panning using method, mentioned in this lesson (cull front faces instead of back faces instead of using bias-value to offset depth values), I got this artefacts as result:

afx → Arccch

7 years ago

Could you please explain what's the difference between your method and the origin method in this lesson that causes the Peter Panning?
Both use front face's depth (with a bias) as closest depth.I guess this is why **@Owen** 's issue occuring.

0 0 Reply

Arccch → afx

7 years ago

Ugh, that was long ago, but as far as I remember the lesson goes through following iterations:
1) Use front faces. Suffer from akne
2) Add bias to front face depth. That fixes akne proplem but Peter Panning suddenly appears
3) And then we finally use front face culling. This way we use back face depth to form the shadow. Now we have nice shadows without akne AND without Peter Panning.

But there's still a problem with the last one: it works fine only for objects with simple geometry. In cases such as mine (with complex human body model) it can cause wierd artifacts(pic. 1) and it does not work for thin (one plane) objects.

To fix that I used front faces (with bias) AND back simultaneously faces to form the shadowmap.

1 0 Reply

Xiaohu Tang → Arccch

2 years ago

You seem to solve the problem perfectly but not explain it clearly. Let me help you explain a bit.

With your algorithm, for each pixel with a depth value in the shadow map, we should have at least two fragments on this pixel if the rendered model has two faces. Then we note these two fragments with frag_front and frag_back which means front face fragment and back face fragment correspondingly.

Then here comes the interesting part. The final depth written to shadow map is min(frag_front_depth + bias, frag_back_depth). If frag_front_depth + bias < frag_back_depth, then the front face is chosen, but the bias is so small that it won't cause peter panning because a even larger frag_back_depth will never cause peter panning. Shadow acne is also avoided because a bias is add to frag_front_depth. If frag_back_depth < frag_front_depth + bias, then the back face is chosen and everything just works fine.

This approach works perfectly for scene that both contains normal "watertight" models and billboards.

1 0 Reply

afx → afx

7 years ago

The only difference maybe appear for very thin object and the the junction of front face and back face, where your method will use back face's depth as the closest depth.

0 0 Reply

MahanGM → Arccch

7 years ago

I think this solution is more acceptable in a real world application scenario.

0 0 Reply

Arccch → Arccch

Anasky → Arccch
8 years ago
Hey guys, I am currently experiencing the exact same issue... Could you please tell me how you solved it?

0 0 Reply

Carlos Peláez González
10 years ago
I think this image shows better the shadow mapping issue.

I don't catch the page image, sorry :P

10 0 Reply

afx → Carlos Peláez González
7 years ago
Disagree. This representation couldn't explain why Peter Panning appears.But if we move the shadow map texel rather than shadowed fragment, it will be more intuitive.
As for the statement"currentDepth - bias > closestDepth ? 1.0 : 0.0; ", move the bias to the right of greater operator then we get currentDepth > closeDepth + bias， corresponding to a picture that move the shadow map texel far away from the light source.

2 0 Reply

Donlod → afx
5 years ago
To make this clear, the image shown in the tutorial combined with the bias code makes the box bigger instead of pushing the yellow lines down? currentDepth - bias makes the value smaller which means its a bit closer to the light which makes it more "likely" that the fragment is not in shadow.

0 0 Reply

俞宏松 → Carlos Peláez González
9 years ago
awesome! This image has a better show for shadow mapping issue. Now does it indicate that if for example setting shadow map resolution is equivalent to Fragment sampling resolution then even though we don`t need Applying shadow bias we will get a exactly correct shadow map?

0 0 Reply

afx → 俞宏松
7 years ago
Nope. Think about different incident angle of light as the lesson mentioned.

1 0 Reply

Sveen
8 years ago
Great

8 0 Reply

NetBeam
2 years ago edited

see more

6 0 Reply

_shinichi
7 years ago
Why are suddenly using the directional light's position?

5 0 Reply

Andrea
7 years ago edited
thank you for the wonderful tutorial!

Model: Stanford bunny
Textures from freepbr.com

5 0 Reply

Random guy

Andrei Despinoiu → Random guy

6 years ago edited

Yeah... Also these two lines (glViewport and glClear) that are called one after another for no particular reason:

```
// reset viewport
glViewport(0, 0, SCR_WIDTH, SCR_HEIGHT);
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

// 2. render scene as normal using the generated depth/shadow map
// --------------------------------------------------------------
glViewport(0, 0, SCR_WIDTH, SCR_HEIGHT); // same as above
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); // same as above
shader.use();
```

0 0 Reply

Illation

9 years ago

Gotta say it nearly feels like cheating looking through your tutorials, you make it way to easy... To be fair I implemented this before in direct x/hlsl. I was wondering if there is something similar to the comparison filter for texture sampling in opengl/glsl

SamplerComparisonState cmpSampler
{
Filter = COMPARISON_MIN_MAG_MIP_LINEAR;
AddressU = MIRROR;
AddressV = MIRROR;
ComparisonFunc = LESS_EQUAL;
};

Apart from that I am going to look into csm next, should be doable as I already implemented frustum culling for a different project.

see more

5 0 Reply

JoeyDeVries Sensei → Illation

9 years ago edited

Man, that looks awesome!

And yes, in OpenGL the equivalent would be:

```
glBindTexture(GL_TEXTURE_2D, textureID);
glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0, format, GL_UNSIGNED_BYTE, data);
glGenerateMipmap(GL_TEXTURE_2D);

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_MIRRORED_REPEAT); // MIRROR is proba
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_MIRRORED_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR); // COMPAR
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

And the depth comparison function is global state which you'll need to set somewhere in your application from where you'd want it to take effect (once set, its state remains):

```
glDepthFunc(GL_LEQUAL);
```

See the Getting-Started/Texture tutorial for some images regarding each texture option. Hope it helps! :)

0 0 Reply

Illation → JoeyDeVries

9 years ago

Hm, well to be honest I was looking for the glsl version of a specific sampler type, and there turns out to be one called "sampler2DShadow", which during the

I guess this would probably be cheaper then using the ternary operator during the PCF calculation.

I think I explained what I was looking for wrong, apologies for that

0 0 Reply

Donlod → Illation
5 years ago edited

Late answer i think you were probably looking for these two

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_COMPARE_MODE, GL_COMPARE_REF_TO_TEXTURE);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_COMPARE_FUNC, GL_LEQUAL);
```

sampling the sampler2DShadow in the fragment shader then looks something like that:

```
float shadowMultiplier = textureProj(shadowMap, shadowCoord);
```

This setup performs the depth comparison as a texture filtering operation, textureProj applies the perspective divide and uses the resulting z component as the REF. shadowMultiplier is 0 if in shadow and 1 if not.You still have to make sure to shift shadowCoord into range [0, 1]. You can do this in the vertex shader using an additional matrix multiplied into the lightSpaceMatrix. It looks somewhat like this:

```
lightSpaceMatrix = shift * light_projection * light_view
with shift = mat4::translate(0.5, 0.5, 0.5) * mat4::scale(0.5) * mat4(1.0)
```

This shift matrix is often called "bias" but it does not have anything to do with removing shadow acne. Applying the bias in clip space instead of after perspective divide results in the same vector except for some floating point inaccuracies.

0 0 Reply

JoeyDeVries Sensei → Illation
9 years ago

Yeah, if I'm not mistaken `sampler2DShadow` checks whether the depth-map's depth is in shadow compared to the current fragment's depth (3rd argument). I'm not sure whether this is written somewhere, but I can imagine it could be beneficial for performance as the drivers could make some specific optimizations there.

I've never used it though, so can't help you much there

0 0 Reply

Illation → JoeyDeVries
9 years ago

Ah, I thought I had commented before, must have forgotten to press post :D
I modified my implementation, and it works just the same but without if statements: https://github.com/Illation...
It also seems to take care of checking the far clipping plane

1 0 Reply

Nights87
11 years ago

Fantastic tutorial once again.
I seem to have encountered an issue though. In the section relating to 'peter panning' you suggest culling the front faces of the objects when rendering to the depth map. When I do this I get the results in the picture. The cube(s) on the ground have a non shadowed area on the part of the floor very close to the cube which should be shadowed. I tried using your source code too and got the same results. Thoughts?

4 0 Reply

JoeyDeVries Sensei → Nights87

the floor's shadow values, but as you can see that isn't always the case.
As with most of shadow mapping it's usually tiny little hacks that don't necessarily work on all objects; in this case it's best not to do face culling when a cube directly touches the ground (or find some other clever trick I can't think of at the top of my head ;))

4 0 Reply

Nights87 JoeyDeVries
11 years ago

Hey Joey,

This kept bugging me so i had another few goes trying to fix it.
I scoured the internet without much luck. Every tutorial i looked at kept saying the best way to solve this issue was to cull front faces when rendering the shadow map, yours included. The thing was, when i culled front faces when rendering the shadow map it created the issue that it was supposed to solve! If I culled back faces instead I didn't have the issue.

I ran some other demo's and they worked as they should so I can confirm its not a weird hardware related problem or anything.

So i kept poking around. Eventually I came across this. If I change
glm::ortho(-10.0f, 10.0f, -10.0f, 10.0f, near_plane, far_plane);
to
glm::ortho(-10.0f, 10.0f, 10.0f, -10.0f, near_plane, far_plane);,

it works correctly. It seems this change flips the shadowmap (?)
In the image below it shows the demo scene with front faces culled when

see more

2 0 Reply

JoeyDeVries Sensei Nights87
11 years ago edited

My guess is that in your original scene you have the culling order (or winding order of the vertices) reversed, in which case you cull the back-faces of all triangles which still results in the obvious edge you see there.

Then my guess is that due to the inversion of the y-axis in the projection matrix, you also inverse the winding order of the vertices rendered, in which case you accidentily fixed the proper faces to cull.

Instead of changing the orthographic projection, try changing the faces to cull (or check that the front faces are properly culled by culling them in your normal scene (with lights and perspective projection)).

You can do this by either setting the faces to cull to `GL_BACK` or changing the 'to-cull-winding-order' of the vertices as follows:

```
glCullFace(GL_BACK);
// or ...
glFrontFace(GL_CW);
```

0 0 Reply

Zhuohan Chen JoeyDeVries
10 years ago

Hey Joey!

At least in my program if I cull the back-faces it cause the Peter Panning and if I cull the front-faces it results in a non-shadowed area very close to the cube.

Besides, why would the cube's back-faces depth map values correspond to the floor's shadow values if I already cull the front-faces?

0 0 Reply

Seems like the bottom of the cube is too close to the floor; when the bias is added the depth of the floor -- the floor's depth is always less than shadow's depth.

With backface culling and a bias value that changes for the angle between the normal and light, I'm not seeing much peter panning.

2 0 Reply

ConsolePeasant92
10 years ago

I want to thank you for these great tutorials, they really helped me creating my game engine! :)
I'm gonna donate you!

3 0 Reply

Azer
2 years ago

2 0 Reply

林泽斌 → Azer
2 years ago

Can you learn your code?

0 0 Reply

SenKuro MaShiro
3 years ago

good.

Max Adams → SenKuro MaShiro
4 months ago
Nice, I used miku as my test bench for lighting effects and .obj file loading.
And your imgui tab reminds me of how much I miss it, writing in C certainly has its downsides lol

0 0 Reply

Dat Pham → SenKuro MaShiro
a year ago
can you give me link download model character please

0 0 Reply

SenKuro MaShiro → Dat Pham
a year ago
Hi, you can find the model here :D
www.aplaybox.com/details/model/balzwHNnXX4z
github.com/zeroruka/GI-Asse...
github.com/HeisensOppings/LearnOpengl_Mingw/tree/main/res/model/lisa

0 0 Reply

Alexander → SenKuro MaShiro
3 years ago
Hey that's a cool scene. I am implementing shadow mapping too and wanted to ask you what shadow resolution you used to draw the scene, if you remember that is.

0 0 Reply

SenKuro MaShiro → Alexander
3 years ago
I'm glad you asked. The scene resolution: SCR_WIDTH = 800 and SCR_HEIGHT = 600. the shadow mapping resolution: SHADOW_WIDTH = SCR_WIDTH * 4 and SHADOW_HEIGHT = SCR_HEIGHT * 4.

0 0 Reply

Alexander → SenKuro MaShiro
3 years ago
Okay, thanks! I just wanted to use it as a reference because I think my light projection matrix covers too much area. Also, I think it is correct to use equal shadow map width and height. Correct me if I'm wrong.

1 0 Reply

SenKuro MaShiro → Alexander
3 years ago
I found that if the scene width and height are not equal, there is no problem if the 2D shadow map has equal length and width. As long as glViewport is set to the width and height of the 2D shadow map when rendering the 2D shadow map, you are right

1 0 Reply

SenKuro MaShiro → Alexander
3 years ago
Yes, if it is a shadow cubemap generated by a point light source, the width and height should be equal

0 0 Reply

satansdildo
4 years ago
In my case, I had to split up this line in the shadow calculation function:

```
projCoords = projCoords * 0.5 + 0.5;
```

I know why it fixes the problem when the calculation is no longer applied to the Z component, my question is why is it NOT broken in this example? For the record I'm using DirectX but all of the math and hardware concepts are the same.

2 0 Reply

Clarence Ng Min Teck
7 years ago

hello i am confuse of this line of code
glUniformMatrix4fv(lightSpaceMatrixLocation, 1, GL_FALSE, glm::value_ptr(lightSpaceMatrix));

where the lightSpaceMatrixLocation variable from?

2 0 Reply

Andrei Despinoiu → Clarence Ng Min Teck
6 years ago edited

An artifact of multiple clean-ups, probably.
The equivalent line from the provided source code is:

```
simpleDepthShader.setMat4("lightSpaceMatrix", lightSpaceMatrix);
```

The "setMat4" method is inside "Shader.h".

Note that these two lines are identical in functionality:

```
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(trans));
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, &trans[0][0]);
```

A few explanations on why they're identical can be found in the comments section from this lesson: https://learnopengl.com/Get...

1 0 Reply

GTruf
2 years ago

For those who are interested in working with shaders, lens flare, atmospheric scattering, working with textures, models and so on, check out my SolarSystem-3D project (don't forget to like it on GitHub).

see more

2 1 Reply

Andreas Wagner
2 years ago

First of all thanks for this nice tutorial series :)

When it comes to the light position, what do I have to consider here? If my light e.g. is at position 0f, 100f, 0f? I assume that I need to adjust the ortho projection matrix accordingly.

1 0 Reply

Load more comments

Subscribe Privacy Do Not Sell My Data

Consent Do Not Sell My Information